

第 5 章-单层网络-分类 – 第 5.4 节-判别分类器

《深度学习基础》课程小组

2026 年 3 月 21 日

目录

1 Discriminative Classifiers	1
1.1 Activation functions	2
1.2 Fixed basis functions	2
1.3 Logistic regression	3
1.3.1 Multi-class logistic regression	4
1.3.2 Probit regression	5
1.3.3 Canonical link functions	7

1 Discriminative Classifiers

For the two-class classification problem, we have seen that the posterior probability of class $\mathcal{C}_1$ can be written as a logistic sigmoid acting on a linear function of $\mathbf{x}$, for a wide choice of class-conditional distributions $p(\mathbf{x}|\mathcal{C}_k)$ from the exponential family. Similarly, for the multi-class case, the posterior probability of class $\mathcal{C}_k$ is given by a softmax transformation of linear functions of $\mathbf{x}$. For specific choices of the class-conditional densities $p(\mathbf{x}|\mathcal{C}_k)$, we have used maximum likelihood to determine the parameters of the densities as well as the class priors $p(\mathcal{C}_k)$ and then used Bayes' theorem to find the posterior class probabilities. This represents an example of *generative modelling*, because we could take such a model and generate synthetic data by drawing values of $\mathbf{x}$ from the marginal distribution $p(\mathbf{x})$ or from any of the class-conditional densities $p(\mathbf{x}|\mathcal{C}_k)$.

However, an alternative approach is to use the functional form of the generalized linear model explicitly and to determine its parameters directly by using maximum likelihood. In this direct approach, we maximize a likelihood function defined through the conditional distribution $p(\mathcal{C}_k|\mathbf{x})$, which represents a form of *discriminative probabilistic modelling*. One advantage of the discriminative approach is that there will typically be fewer learnable parameters to be determined, as we will see shortly. It may also lead to improved predictive performance, particularly when the assumed forms for the class-conditional densities represent a poor approximation to the true distributions.

1.1 Activation functions

In linear regression, the model prediction $y(\mathbf{x}, \mathbf{w})$ is given by a linear function of the parameters

$$y(\mathbf{x}, \mathbf{w}) = \mathbf{w}^T \mathbf{x} + w_0, \quad (5.69)$$

which gives a continuous-valued output in the range $(-\infty, \infty)$. For classification problems, however, we wish to predict discrete class labels, or more generally posterior probabilities that lie in the range $(0, 1)$. To achieve this, we consider a generalization of this model in which we transform the linear function of $\mathbf{w}$ and w_0 using a nonlinear function $f(\cdot)$ so that

$$y(\mathbf{x}, \mathbf{w}) = f(\mathbf{w}^T \mathbf{x} + w + 0). \quad (5.70)$$

In the machine learning literature, $f(\cdot)$ is known as an *activation function*, whereas its inverse is called a *link function* in the statistics literature. The decision surfaces correspond to $y(\mathbf{x}) = \text{constant}$, so that $\mathbf{w}^T \mathbf{x} = \text{constant}$, and hence the decision surfaces are linear functions of $\mathbf{x}$, even if the function $f(\cdot)$ is nonlinear. For this reason, the class of models described by (5.70) are called *generalized linear models* (McCullagh and Nelder, 1989). However, in contrast to the models used for regression, they are no longer linear in the parameters due to the nonlinear function $f(\cdot)$. This will lead to more complex analytical and computational properties than for linear regression models. Nevertheless, these models are still relatively simple compared to the much more flexible nonlinear models that will be studied in subsequent chapters.

1.2 Fixed basis functions

So far in this chapter, we have considered classification models that work directly with the original input vector $\mathbf{x}$. However, all the algorithms are equally applicable if we first make a fixed nonlinear transformation of the inputs using a vector of basis functions $\phi(\mathbf{x})$. The resulting decision boundaries will be linear in the feature space ϕ , and these correspond to nonlinear decision boundaries in the original $\mathbf{x}$ space, as illustrated in Figure 5.15. Classes that are linearly separable in the feature space $\phi(\mathbf{x})$ need not be linearly separable in the original observation space $\mathbf{x}$.

Note that as in our discussion of linear models for regression, one of the basis functions is typically set to a constant, say $\phi_0(\mathbf{x}) = 1$, so that the corresponding parameter w_0 plays the role of a bias.

For many problems of practical interest, there is significant overlap in $\mathbf{x}$ -space between the class-conditional densities $p(\mathbf{x}|\mathcal{C}_k)$. This corresponds to posterior probabilities $p(\mathcal{C}_k|\mathbf{x})$, which, for at least some values of $\mathbf{x}$, are not 0 or 1. In such cases, the optimal solution is obtained by modelling the posterior probabilities accurately and then applying standard decision theory. Note that nonlinear transformations $\phi(\mathbf{x})$ cannot remove such a class overlap, although they can increase the level of overlap or create an overlap where none existed in the original observation space. However, suitable choices of nonlinearity can make the process of modelling the posterior probabilities easier. However, such fixed basis function models have important limitations, and these will be resolved in later chapters by allowing the basis functions themselves to adapt to the data.

1.3 Logistic regression

We first consider the problem of two-class classification. In our discussion of generative approaches in Section 5.3, we saw that under rather general assumptions, the posterior probability of class $\mathcal{C}_1$ can be written as a logistic sigmoid acting on a linear function of the feature vector ϕ so that

$$p(\mathcal{C}_1|\phi) = y(\phi) = \sigma(\mathbf{w}^T \phi) \quad (5.71)$$

with $p(\mathcal{C}_2|\phi) = 1 - p(\mathcal{C}_1|\phi)$. Here $\sigma(\cdot)$ is the *logistic sigmoid function* defined by (5.42). In the terminology of statistics, this model is known as *logistic regression*, although it should be emphasized that this is a model for classification rather than for continuous variable.

For an M -dimensional feature space ϕ , this model has M adjustable parameters. By contrast, if we had fitted Gaussian class-conditional densities using maximum likelihood, we would have used $2M$ parameters for the means and $M(M+1)/2$ parameters for the (shared) covariance matrix. Together with the class prior $p(\mathcal{C}_1)$, this gives a total of $M(M+5)/2 + 1$ parameters, which grows quadratically with M , in contrast to the linear dependence on M of the number of parameters in logistic regression. For large values of M , there is a clear advantage in working with the logistic regression model directly.

We now use maximum likelihood to determine the parameters of the logistic regression model. To do this, we will make use of the derivative of the logistic sigmoid function, which can conveniently be expressed in terms of the sigmoid function itself:

$$\frac{d\sigma}{da} = \sigma(1 - \sigma). \quad (5.72)$$

For a data set $\{\phi_n, t_n\}$, where $\phi_n = \phi(\mathbf{x}_n)$ and $t_n \in \{0, 1\}$, with $n = 1, \dots, N$, the likelihood function can be written

$$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^N y_n^{t_n} \{1 - y_n\}^{1-t_n} \quad (5.73)$$

where $\mathbf{t} = (t_1, \dots, t_N)^T$ and $y_n = p(\mathcal{C}_1|\phi_n)$. As usual, we can define an error function by taking the negative logarithm of the likelihood, which gives the *cross-entropy error function*:

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\} \quad (5.74)$$

where $y_n = \sigma(a_n)$ and $a_n = \mathbf{w}^T \phi_n$. Taking the gradient of the error function with respect to $\mathbf{w}$, we obtain

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n \quad (5.75)$$

where we have made use of (5.72). We see that the factor involving the derivative of the logistic sigmoid has cancelled, leading to a simplified form for the gradient of the log likelihood. In particular, the contribution to the gradient from data point n is given by the 'error' $y_n - t_n$ between the target value and the prediction of the model times the basis function vector ϕ_n . Furthermore, comparison

with (4.12) shows that this takes precisely the same form as the gradient of the sum-of-squares error function for the linear regression model.

The maximum likelihood solution corresponds to $\nabla E(\mathbf{w}) = 0$. However, from (5.75) we see that this no longer corresponds to a set of linear equations, due to the nonlinearity in $y(\cdot)$, and so this equation does not have a closed-form solution. One approach to finding a maximum likelihood solution would be to use stochastic gradient descent, in which ∇E_n is the n th term on the right-hand side of (5.75). Stochastic gradient descent will be the principal approach to training the highly nonlinear neural networks discussed in later chapters. However, the maximum likelihood equation is only 'slightly' nonlinear, and in fact the error function (5.74), in which the model is defined by (5.71), is a convex function of the parameters, which allows the error function to be minimized using a simple algorithm called *iterative reweighted least squares* or IRLS (Bishop, 2006). However, this does not easily generalize to more complex models such as deep neural networks.

Note that maximum likelihood can exhibit severe over-fitting for data sets that are linearly separable. This arises because the maximum likelihood solution occurs when the hyperplane corresponding to $\sigma = 0.5$, equivalent to $\mathbf{w}^T \phi = 0$, separates the two classes and the magnitude of $\mathbf{w}$ goes to infinity. In this case, the logistic sigmoid function becomes infinitely steep in feature space, corresponding to a Heaviside step function, so that every training point from each class k is assigned a posterior probability $p(C_k|\mathbf{x}) = 1$. Furthermore, there is typically a continuum of such solutions because any separating hyperplane will give rise to the same posterior probabilities at the training data points. Maximum likelihood provides no way to favour one such solution over another, and which solution is found in practice will depend on the choice of optimization algorithm and on the parameter initialization. Note that the problem will arise even if the number of data points is large compared with the number of parameters in the model, so long as the training data set is linearly separable. The singularity can be avoided by adding a regularization term to the error function.

1.3.1 Multi-class logistic regression

In our discussion of generative models for multi-class classification, we have seen that, for a large class of distributions from the exponential family, the posterior probabilities are given by a softmax transformation of linear functions of the feature variables, so that

$$p(C_k|\phi) = y_k(\phi) = \frac{\exp(a_k)}{\sum_j \exp(a_j)} \quad (5.76)$$

where the pre-activations a_k are given by

$$a_k = \mathbf{w}_k^T \phi. \quad (5.77)$$

There we used maximum likelihood to determine separately the class-conditional densities and the class priors and then found the corresponding posterior probabilities using Bayes' theorem, thereby implicitly determining the parameters $\{\mathbf{w}_k\}$. Here we consider the use of maximum likelihood to determine the parameters $\{\mathbf{w}_k\}$ of this model directly. To do this, we will require the derivatives of y_k with respect to all of the pre-activations a_j . These are given by

$$\frac{\partial y_k}{\partial a_j} = y_k(I_{kj} - y_j) \quad (5.78)$$

where I_{kj} are the elements of the identity matrix.

Next we write down the likelihood function. This is most easily done using the 1-of- K coding scheme in which the target vector $\mathbf{t}_n$ for a feature vector ϕ_n belonging to class $\mathcal{C}_k$ is a binary vector with all elements zero except for element k , which equals one. The likelihood function is then given by

$$p(\mathbf{T}|\mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K p(\mathcal{C}_k|\phi_n)^{t_{nk}} = \prod_{n=1}^N \prod_{k=1}^K y_{nk}^{t_{nk}} \quad (5.79)$$

where $y_{nk} = y_k(\phi_n)$, and $\mathbf{T}$ is an $N \times K$ matrix of target variables with elements t_{nk} . Taking the negative logarithm then gives

$$E(\mathbf{w}_1, \dots, \mathbf{w}_K) = -\ln p(\mathbf{T}|\mathbf{w}_1, \dots, \mathbf{w}_K) = -\sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}, \quad (5.80)$$

which is known as the *cross-entropy error function* for the multi-class classification problem.

We now take the gradient of the error function with respect to one of the parameter vectors $\mathbf{w}_j$. Making use of the result (5.78) for the derivatives of the softmax function, we obtain

$$\nabla_{\mathbf{w}_j} E(\mathbf{w}_1, \dots, \mathbf{w}_K) = \sum_{n=1}^N (y_{nj} - t_{nj}) \phi_n \quad (5.81)$$

where we have made use of $\sum_k t_{nk} = 1$. Again, we could optimize the parameters through stochastic gradient descent.

Once again, we see the same form arising for the gradient as was found for the sum-of-squares error function with the linear model and for the cross-entropy error with the logistic regression model, namely the product of the error $(y_{nj} - t_{nj})$ times the basis function activation ϕ_n . These are examples of a more general result that we will explore later.

Linear classification models can be represented as single-layer neural networks as shown in Figure 5.16. If we consider the derivative of the error function with respect to a weight w_{ik} , which links basis function $\phi_i(\mathbf{x})$ to output unit t_k , we have from (5.81)

$$\frac{\partial E(\mathbf{w}_1, \dots, \mathbf{w}_K)}{\partial w_{ij}} = \sum_{n=1}^N (y_{nk} - t_{nk}) \phi_i(\mathbf{x}_n). \quad (5.82)$$

Comparing this with Figure 5.16, we see that, for each data point n this gradient takes the form of the output of the basis function at the input end of the weight link with the 'error' $(y_{nk} - t_{nk})$ at the output end.

1.3.2 Probit regression

We have seen that, for a broad range of class-conditional distributions described by the exponential family, the resulting posterior class probabilities are given by a logistic (or softmax) transformation

acting on a linear function of the feature variables. However, not all choices of class-conditional density give rise to such a simple form for the posterior probabilities, which suggests that it might be worth exploring other types of discriminative probabilistic model. Consider the two-class case, again remaining within the framework of generalized linear models, so that

$$p(t = 1|a) = f(a) \quad (5.83)$$

where $a = \mathbf{w}^T \boldsymbol{\phi}$, and $f(\cdot)$ is the activation function.

One way to motivate an alternative choice for the link function is to consider a noisy threshold model, as follows. For each input $\boldsymbol{\phi}_n$, we evaluate $a_n = \mathbf{w}^T \boldsymbol{\phi}_n$ and then we set the target value according to

$$\begin{cases} t_n = 1, & \text{if } a_n \geq \theta, \\ t_n = 0, & \text{otherwise.} \end{cases} \quad (5.84)$$

If the value of θ is drawn from a probability density $p(\theta)$, then the corresponding activation function will be given by the cumulative distribution function

$$f(a) = \int_{-\infty}^a p(\theta) d\theta \quad (5.85)$$

as illustrated in Figure 5.17.

As a specific example, suppose that the density $p(\theta)$ is given by a zero-mean, unit-variance Gaussian. The corresponding cumulative distribution function is given by

$$\Phi(a) = \int_{-\infty}^a \mathcal{N}(\theta|0, 1) d\theta, \quad (5.86)$$

which is known as the *probit function*. It has a sigmoidal shape and is compared with the logistic sigmoid function in Figure 5.12. Note that the use of a Gaussian distribution with general mean and variances does not change the model because this is equivalent to a re-scaling of the linear coefficients $\mathbf{w}$. Many numerical packages can evaluate a closely related function defined by

$$\text{erf}(a) = \frac{2}{\sqrt{\pi}} \int_0^a \exp(-\theta^2/2) d\theta \quad (5.87)$$

and known as the *erf function* or *error function* (not to be confused with the error function of a machine learning model). It is related to the probit function by

$$\Phi(a) = \frac{1}{2} \left\{ 1 + \frac{1}{\sqrt{2}} \text{erf}(a) \right\}. \quad (5.88)$$

The generalized linear model based on a probit activation function is known as *probit regression*. We can determine the parameters of this model using maximum likelihood by a straightforward extension of the ideas discussed earlier. In practice, the results found using probit regression tend to be like those of logistic regression.

One issue that can occur in practical applications is that of *outliers*, which can arise for instance through errors in measuring the input vector $\mathbf{x}$ or through mislabelling of the target value t . Because such points can lie a long way to the wrong side of the ideal decision boundary, they can seriously distort the classifier. The logistic and probit regression models behave differently in this respect because the tails of the logistic sigmoid decay asymptotically like $\exp(-x)$ for $|x| \rightarrow \infty$, whereas for the probit activation function, they decay like $\exp(-x^2)$, and so the probit model can be significantly more sensitive to outliers.

1.3.3 Canonical link functions

For the linear regression model with a Gaussian noise distribution, the error function, corresponding to the negative log likelihood, is given by (4.11). If we take the derivative with respect to the parameter vector $\mathbf{w}$ of the contribution to the error function from a data point n , this takes the form of the 'error' $y_n - t_n$ times the feature vector ϕ_n , where $y_n = \mathbf{w}^T \phi_n$. Similarly, for the combination of the logistic-sigmoid activation function and the cross-entropy error function (5.74) and for the softmax activation function with the multi-class cross-entropy error function (5.80), we again obtain this same simple form. We now show that this is a general result of assuming a conditional distribution for the target variable from the exponential family along with a corresponding choice for the activation function known as the *canonical link function*.

We again make use of the restricted form (3.169) of exponential family distributions. Note that here we are applying the assumption of exponential family distribution to the target variable t , in contrast to Section 5.3.4 where we applied it to the input vector $\mathbf{x}$. We therefore consider conditional distributions of the target variable of the form

$$p(t|\eta, s) = \frac{1}{s} h\left(\frac{t}{s}\right) g(\eta) \exp\left\{\frac{\eta t}{s}\right\}. \quad (5.89)$$

Using the same line of argument as led to the derivation of the result (3.172), we see that the conditional mean of t , which we denote by y , is given by

$$y \equiv \mathbb{E}[t|\eta] = -s \frac{d}{d\eta} \ln g(\eta). \quad (5.90)$$

Thus, y and η are simply related, and we denote this relation through $\eta = \psi(y)$.

Following Nelder and Wedderburn (1972), we define a *generalized linear model* to be one for which y is a nonlinear function of a linear combination of the input (or feature) variables so that

$$y = f(\mathbf{w}^T \phi) \quad (5.91)$$

where $f(\cdot)$ is known as the activation function in the machine learning literature, and $f^{-1}(\cdot)$ is known as the link function in statistics.

Now consider the log likelihood function for this model, which, as a function of η , is given by

$$\ln p(\mathbf{t}|\eta, s) = \sum_{n=1}^N \ln p(t_n|\eta, s) = \sum_{n=1}^N \left\{ \ln g(\eta_n) + \frac{\eta_n t_n}{s} \right\} + \text{const} \quad (5.92)$$

where we are assuming that all observations share a common scale parameter (which corresponds to the noise variance for a Gaussian distribution, for instance) and so s is independent of n . The derivative of the log likelihood with respect to the model parameters $\mathbf{w}$ is then given by

$$\begin{aligned}\nabla_{\mathbf{w}} \ln p(\mathbf{t}|\eta, s) &= \sum_{n=1}^N \left\{ \frac{d}{d\eta_n} \ln g(\eta_n) + \frac{t_n}{s} \right\} \frac{d\eta_n}{dy_n} \frac{dy_n}{da_n} \nabla_{\mathbf{w}} a_n \\ &= \sum_{n=1}^N \frac{1}{s} \{t_n - y_n\} \psi'(y_n) f'(a_n) \phi_n\end{aligned}\tag{1}$$

where $a_n = \mathbf{w}^T \phi_n$, and we have used $y_n = f(a_n)$ together with the result (5.90) for $\mathbb{E}[t|\eta]$. We now see that there is a considerable simplification if we choose a particular form for the link function $f^{-1}(y)$ given by

$$f^{-1}(y) = \psi(y),\tag{5.94}$$

which gives $f(\psi(y)) = y$ and hence $f'(\psi)\psi'(y) = 1$. Also, because $a = f^{-1}(y)$, we have $a = \psi$ and hence $f'(a)\psi'(y) = 1$. In this case, the gradient of the error function reduces to

$$\nabla \ln E(\mathbf{w}) = \frac{1}{s} \sum_{n=1}^N \{y_n - t_n\} \phi_n.\tag{5.95}$$

We have seen that there is a natural pairing between the choice of error function and the choice of output-unit activation function. Although we have derived this result in the context of single-layer network models, the same considerations apply to deep neural networks discussed in later chapters.